

SynapseGrid — Healthcare Management System

Project Report

Name: Viketh Hegde **Cohort:** Mark Zuckerberg **Roll No.:** 150096724071

1. Introduction

Modern healthcare operations demand robust, scalable software capable of managing patient records, appointments, doctor information, medical inventory, and emergency services within a unified platform. Traditional systems frequently suffer from fragmented data management, inconsistent deployment practices, and limited operational visibility.

SynapseGrid addresses these challenges by combining a modular, REST-based healthcare application with a structured DevOps pipeline. By integrating containerization, continuous integration, container orchestration, and monitoring into the development lifecycle, SynapseGrid delivers both practical healthcare functionality and a replicable foundation for scalable software deployment.

2. Problem Statement

Healthcare management systems must coordinate multiple interconnected services — patient registration, appointment scheduling, doctor management, pharmacy operations, and emergency response — while ensuring high availability and data integrity. Manual deployment and configuration introduce operational complexity, increase the risk of service disruptions, and make consistent environment replication difficult.

This project addresses these challenges by developing a healthcare platform underpinned by DevOps methodologies, enabling systematic deployment, monitoring, and maintenance across the full application lifecycle.

3. Objectives

The project aims to:

- Develop a healthcare management application covering patient, doctor, appointment, pharmacy, and emergency services.
 - Persist and retrieve records using PostgreSQL.
 - Expose application functionality through a REST API built on Flask.
 - Containerize the application using Docker and manage multi-service coordination with Docker Compose.
 - Automate build and integration processes using Jenkins.
 - Demonstrate container orchestration using Kubernetes.
 - Monitor system components using Prometheus.
 - Incorporate secure infrastructure management using Vault and Terraform.
 - Demonstrate a complete, end-to-end DevOps workflow.
-

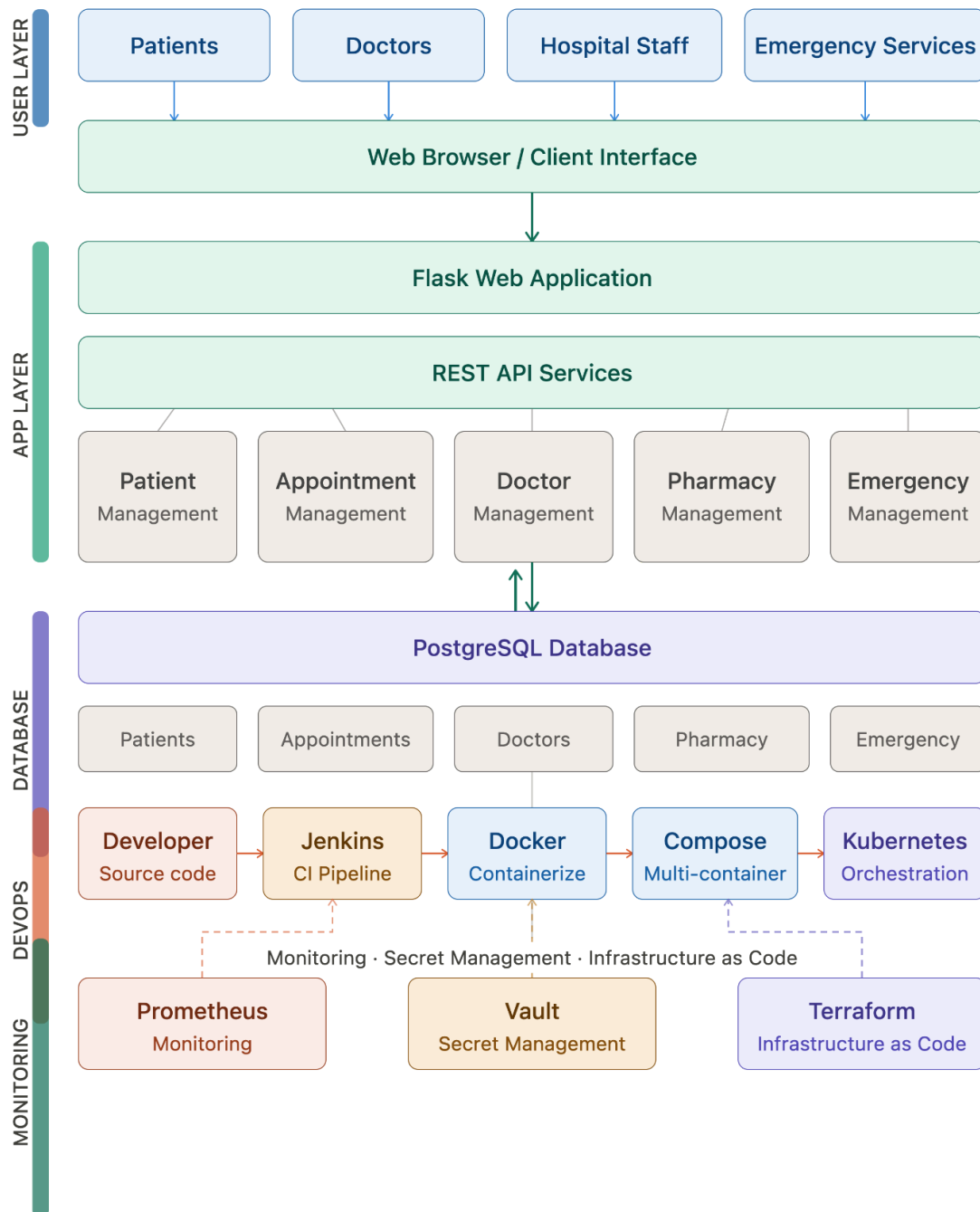
4. Proposed Solution

SynapseGrid combines healthcare management with a modern DevOps infrastructure. Flask provides REST API endpoints for healthcare services, while PostgreSQL ensures reliable data storage. Docker and Docker Compose simplify containerization and multi-service deployment, and Jenkins automates the CI/CD pipeline. Kubernetes manages application orchestration, while Prometheus monitors system performance. Together, these components create a modular and scalable platform that integrates software engineering and DevOps practices, with each service independently manageable and extensible for future growth.

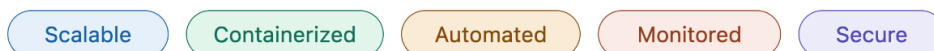
5. System Architecture

SynapseGrid is structured into five layers:

- **User Layer:** Patients, doctors, hospital staff, and emergency services access the system through a web interface.
- **Application Layer:** A Flask-based REST API handles requests and manages patient, doctor, appointment, pharmacy, and emergency modules.
- **Database Layer:** PostgreSQL stores all application data and supports bidirectional communication with Flask.
- **DevOps Layer:** Jenkins, Docker, Docker Compose, and Kubernetes automate CI/CD, containerization, and deployment.
- **Monitoring & Infrastructure Layer:** Prometheus monitors system health, Vault secures secrets, and Terraform provisions infrastructure.



SynapseGrid Healthcare Platform



6. Functional Modules

6.1 Patient Management

Handles registration and retrieval of patient records. Exposes REST endpoints for creating and querying patient data, with records referenced by appointment and emergency modules.

6.2 Appointment Management

Manages scheduling and retrieval of appointments between patients and doctors, enforcing referential integrity at the database level.

6.3 Doctor Management

Stores doctor profiles including specialisation and availability. Doctor records are referenced by appointments and queryable independently through the API.

6.4 Pharmacy Management

Manages medication and inventory records, supporting queries on available medications and stock levels.

6.5 Emergency Management

Records and retrieves emergency cases linked to patient records, supporting rapid data entry and lookup appropriate to time-sensitive scenarios.

6.6 Database Connectivity

All modules share a central database connection layer. Connection parameters are managed via configuration, allowing environment-specific values without code changes.

6.7 Monitoring Services

Prometheus scrapes metrics endpoints exposed by the Flask application, providing visibility into request rates, error rates, and system resource usage.

6.8 Infrastructure Management

Vault injects secrets at runtime for database credentials and API keys. Terraform provides reproducible infrastructure definitions for environment provisioning.

7. Database Design

The PostgreSQL database uses a dedicated table per domain. Schema simplicity is intentional to keep focus on end-to-end system behaviour.

Table	Key Fields
patients	patient_id, name, date_of_birth, contact
doctors	doctor_id, name, specialisation, contact
appointments	appointment_id, patient_id, doctor_id, date, status
pharmacy	item_id, name, quantity, unit
emergency	case_id, patient_id, description, severity, timestamp

Foreign key relationships link appointments and emergency records to the patients and doctors tables.

8. API Architecture

The Flask application exposes a REST API organised by domain. All endpoints return JSON responses.

Method	Endpoint	Description
GET	/patients	Retrieve all patient records
GET	/patients/<id>	Retrieve a specific patient
GET	/doctors	Retrieve all doctor records
GET	/appointments	Retrieve all appointments
GET	/pharmacy	Retrieve pharmacy inventory
GET	/emergency	Retrieve emergency cases
GET	/health	Application health check

The `/health` endpoint is consumed by Prometheus for liveness monitoring.

9. Technology Stack

Flask — Lightweight REST API framework. Its simplicity keeps focus on infrastructure concerns without excess application complexity.

PostgreSQL — Production-grade relational database with strong container support, used for all persistent storage.

Docker — Packages the application and dependencies into portable container images, ensuring consistency across environments.

Docker Compose — Declaratively manages multi-container startup, networking, and volume configuration with a single command.

Jenkins — Automates the build pipeline on each code change, producing consistent, auditable build artefacts.

Kubernetes — Demonstrates declarative container orchestration, self-healing, and scaling of application workloads.

Prometheus — Collects and stores time-series metrics from the application for operational monitoring.

Vault — Manages secrets injection at runtime, keeping credentials out of source code and container images.

Terraform — Expresses infrastructure as code, enabling reproducible and auditable environment provisioning.

10. DevOps Pipeline

The SynapseGrid pipeline follows a linear, stage-gated workflow:

1. **Developer** commits source code to the repository.
2. **Jenkins** detects the commit and triggers the CI pipeline.
3. **Docker Build** produces a container image of the Flask application.
4. **Docker Compose** brings up the full multi-container environment.
5. **Kubernetes** manages orchestration and lifecycle of the running containers.
6. **Prometheus** begins scraping metrics from the live application.

Each stage depends on the success of the prior stage, ensuring only validated builds progress through the pipeline.

11. Individual DevOps Components

11.1 Docker

The Dockerfile defines the base image, dependency installation, and application entry point. The resulting image behaves identically across development and production environments.

11.2 Docker Compose

The `docker-compose.yml` defines three services: Flask application, PostgreSQL database, and Prometheus. Compose manages inter-service networking, volume mounts, and startup ordering.

11.3 Jenkins

The Jenkinsfile defines a multi-stage pipeline covering source checkout, Docker image build, and post-build validation, providing an automated and auditable build process.

11.4 Kubernetes

Kubernetes manifests define a Deployment and a Service for the Flask application. The Deployment maintains the desired replica count and provides self-healing by restarting failed containers.

11.5 Prometheus

Prometheus scrapes the `/metrics` endpoint at defined intervals, collecting HTTP request counts and latency data, with targets configured in `prometheus.yml`.

11.6 Vault

Database credentials and API keys are stored in Vault and injected at runtime via environment variables, preventing sensitive values from appearing in source code or container images.

11.7 Terraform

Terraform configuration files declaratively define the infrastructure environment, enabling consistent teardown and reprovisioning for both development and production use.

12. Testing and Validation

Test Area	Approach
Functional testing	API endpoints tested for expected responses across all five modules

Database connectivity	Verified reads and writes against PostgreSQL for all tables
Container validation	Docker images confirmed to build and run without errors
Jenkins build	Pipeline executed end-to-end; build artefacts verified
Kubernetes deployment	Pods confirmed running; service endpoints accessible
Prometheus monitoring	Metrics endpoint scraped; data visible in Prometheus UI
API endpoint validation	All endpoints return correct HTTP status codes and JSON payloads

13. Performance and Reliability

Containerisation ensures environment consistency and isolates each service with defined resource boundaries.

Modular architecture allows individual modules to be updated without impacting unrelated services.

Fault tolerance is addressed through Kubernetes' self-healing, which automatically restarts failed pods to maintain the desired replica count.

Maintainability is improved through infrastructure-as-code practices — Terraform and Compose configurations serve as living documentation of the environment.

Deployment efficiency is achieved via the Jenkins pipeline, reducing time between a code commit and a running, validated application.

14. Future Scope

- **Authentication and authorisation** — Role-based access control for patients, doctors, and administrative staff.
 - **Cloud deployment** — Deployment to a managed cloud provider with auto-scaling and managed database services.
 - **Mobile integration** — Native mobile applications for patient-facing functionality.
 - **AI-assisted services** — Diagnostic assistance and predictive pharmacy inventory management.
 - **Advanced monitoring** — Grafana integration with Prometheus for richer operational dashboards.
 - **Microservice decomposition** — Splitting the monolith into independently deployable per-domain services.
 - **Real-time notifications** — Event-driven alerts for appointments, emergencies, and system health.
-

15. Conclusion

SynapseGrid demonstrates the integration of healthcare application development with a complete DevOps methodology. The project implements patient, doctor, appointment, pharmacy, and emergency management within a Flask REST API backed by PostgreSQL, while containerizing, automating, orchestrating, and monitoring the application through Docker, Jenkins, Kubernetes, and Prometheus.

The platform serves as both a functional healthcare tool and a practical reference for DevOps pipeline design, illustrating how modern infrastructure practices produce a scalable, maintainable, and operationally visible system.